

Python library hijacking

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un script de Python que se ejecuta como root hace 'import modulo', Python busca ese modulo recorriendo una lista de directorios (sys.path) en orden. Si puedes colocar un fichero .py con ese nombre en un directorio que se busque ANTES que el modulo real, o sobrescribir el propio modulo, tu codigo se ejecuta con privilegios de root al importarse. Es el path-hijacking aplicado al sistema de imports de Python.

sys.path incluye, y normalmente en primer lugar, el directorio del propio script; ademas de PYTHONPATH y los site-packages. El programador confia en que se importara el modulo legitimo, pero Python simplemente coge el primero que encuentra. Si el directorio del script es escribible, o el modulo importado lo es, o sudo conserva PYTHONPATH, rompes esa confianza. Misma logica que el privesc-modelo.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando puedes ejecutar un script Python como root (via sudo-abuse) o un cron root lo ejecuta, Y puedes escribir uno de los sitios donde Python busca sus modulos. Es un vector muy comun en rooms de escalada Linux.

Cómo identificarla

- sudo -l te deja ejecutar un script .py como root.
- El directorio del script (o un modulo que importa) es escribible por tu usuario.
- El script hace 'import' de un modulo con nombre poco comun que vive junto a el.
- sudo -l muestra env_keep+=PYTHONPATH.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Revisa sys.path y los permisos de cada directorio anterior al modulo real que importa el script.

Evidencia esperada: Un directorio escribible por ti apareciendo ANTES del modulo legitimo en sys.path.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
sudo -l
```

Objetivo y contexto: Punto de partida: ver si puedes ejecutar un script .py como root. Una entrada NOPASSWD sobre un python es el candidato típico de este vector.

Resultado esperado: Líneas '(root) NOPASSWD: /usr/bin/python3 /ruta/script.py'. Abre y lee ese script: te dice que módulos importa y donde vive.

Capa 2: prueba documentada

```
python3 -c 'import sys; print(sys.path)'
```

Objetivo y contexto: Muestra el orden en que Python busca módulos. El primer directorio escribible que preceda al módulo real es tu hueco para inyectar.

Resultado esperado: Lista de directorios. Si el dir del script (o " = dir actual) va primero y es escribible, tienes hijacking directo. Cruza esto con los permisos.

Capa 3: prueba documentada

```
echo 'import os; os.setuid(0); os.system("/bin/bash")' > /dir_escribible/modulo.py
```

Objetivo y contexto: Crea un módulo malicioso con el MISMO nombre que el que importa el script. Al ejecutarse el script como root, importa el tuyo y ejecuta tu código con uid 0.

Resultado esperado: Al lanzar 'sudo python3 /ruta/script.py', obtienes una shell root (id = uid=0). Ajusta el nombre del fichero al módulo exacto que importa el script.

Anatomía de los payloads

La primera prueba es `sudo -l`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un directorio escribible por ti apareciendo ANTES del módulo legítimo en `sys.path`.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lee el script root e identifica que modulos importa.
- Mira sys.path y los permisos: ¿puedes escribir el dir del script, el modulo, o un dir que preceda al real?
- Crea un fichero con el nombre del modulo importado que ejecute tu payload (os.setuid(0) + shell).
- Ejecuta el script como root; al importar tu modulo, corre como root. Confirma con id (uid=0).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El script root no hace ningun 'import' de un modulo no estandar.
- Todos los directorios de sys.path anteriores al modulo real son de solo-lectura para ti.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.

3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un script Python que corre como root importa un modulo; si puedes escribir ese modulo (o uno que Python busque antes), tu código corre como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (python-library-hijacking)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.